

GordianBench: Mechanism Predicts Repair Success Better Than Model Tier

A chaos-graded benchmark for agentic distributed-systems debugging

Final Research Report · Phase 4 foundations through Phase 6 capability sweep · 2026-07-01

ABSTRACT

Software-repair benchmarks usually reduce correctness to a visible test passing, which rewards fixes that hold under nominal timing but break under realistic distributed uncertainty. GordianBench replaces that with a chaos-graded oracle: a patch counts as a fix only if it preserves the read-your-writes invariant across repeated trials under injected replication lag, defeating symptom-suppressing “band-aid” repairs. Using this oracle we ran a pre-registered, frozen-configuration k-shot sweep (k=5) of five models on a single distributed bug — a stale-read-after-failover in a primary/replica ledger service.

The result is not a leaderboard. Under identical conditions the five cells produce resolution rates of 0%, 0%, 40%, 40%, and 100%. Read as a tier story this is a loose gradient, but that reading breaks in the middle: a mid-tier model (GPT-4.1) ties a frontier model (Opus 4.8) at 40%, crossing the tier boundary. Patch-level inspection explains why, and yields the paper’s central finding: **the mechanism a model reaches for predicts success better than its tier**. Every one of the nine passing runs, across four different models, used one of three sound mechanisms (always-read-primary, value-watermark catch-up, or post-commit WAL-LSN capture). Every failure either produced no correct mechanism or reached for the canonical LSN fix and mis-implemented one specific transaction-ordering detail. The frontier model Opus failed on 3 of 5 runs *because* it attempted the sophisticated LSN mechanism and captured the WAL position inside the writing statement (a pre-commit watermark), whereas the simpler always-primary mechanism used by lower-tier passes is correct by construction. Sophistication, mis-executed, underperformed bluntness.

1. Background and Motivation

Traditional repair benchmarks collapse correctness onto visible test pass/fail. In distributed systems this is dangerous: a fix can pass every test under calm timing and still violate its invariant the moment real latency, reordering, or partition appears. Worse, an agent can *suppress the symptom* — bump a timeout, add a sleep, retry a read — in ways that pass a naive test while leaving the underlying race intact. A benchmark that cannot tell a genuine invariant-preserving repair from a timing-luck band-aid cannot measure debugging ability at all.

GordianBench is built around a stronger requirement: a fix must survive controlled chaos and preserve its invariant across repeated trials. The contribution is the measurement protocol, not a new scoreboard. It was developed through staged gates, each closed before the next opened: **Phase 4** established a trusted agent loop and two-tier oracle semantics; **Phase 5** hardened the instrument (a correctness-aware legibility classifier, a deterministic patch-delivery pipeline, and a chaos oracle validated against a band-aid corpus); and **Phase 6** ran the frozen-configuration capability sweep reported here.



$$PASS = 0 \text{ stale reads across } 10 \text{ oracle trials} \cdot \text{resolution rate} = PASS / \text{scored runs}$$

Figure 1. The Phase 6 measurement pipeline. Each model run edits a workspace; the workspace diff is the patch; a build check gates compilation; the Tier-2 oracle grades read-your-writes under recovering lag across 10 trials; and the rollup scores from oracle verdicts only.

2. Methods

2.1 Archetype and oracle

The evaluation archetype is **stale-read-after-failover**: a ledger service backed by a primary and a read replica. A deposit commits to the primary; a subsequent balance read may be served by a replica that has not yet replayed the write. The invariant is read-your-writes: an acknowledged write must be visible to the next read.

The Tier-2 oracle injects a **recovering-lag** chaos profile — a 20-second downstream WAL latency toxic, so the replica is perpetually behind on recent writes but eventually catches up. This is the canonical transient-lag manifestation, chosen because it admits multiple correct fix families while rejecting every fixed-wait band-aid: a 20-second lag outlasts any hard-coded sleep or TTL. A patch is graded across 10 trials; **PASS** requires zero stale reads. The oracle was validated GREEN against a band-aid corpus (TTL-bump, sleep-before-read, retry, and a pre-commit-LSN fixture), each correctly failed, and against the reference fix, correctly passed.

2.2 Scoring and discipline

Oracle verdicts only. Resolution rates come from oracle grades; a static classifier labels each patch’s mechanism for diagnostics but never contributes to scoring — across the sweep, classifier-oracle agreement was only 16/25, which is precisely why it is excluded from the metric. **Frozen configuration:** every cell used identical settings (Table 1); no knob moved between cells. **Pre-registered rerun policy:** only infrastructure failures were eligible for rerun; capability, build, and no-patch outcomes were reported as-is, with all pre-commitments fixed in writing before any sweep number existed. **A held-out model:** Gemini 3.1 Pro was excluded (not scored zero) because under the shared prompt it investigated but did not deliver patches — an explicit omission, not a capability result.

3. Building a Trustworthy Instrument (Phases 4-5)

Phase 4 established that the harness could grade real model trajectories end-to-end, and introduced the safety/liveness decomposition: on an earlier archetype (a non-idempotent payment retry), models reliably repaired the source-legible half of a bug while missing a runtime-discoverable half, producing a safety-pass/liveness-fail signature. This drove the design of the stale-read archetype to *force runtime investigation* rather than reward source-only pattern-matching — a property later confirmed, with no model resolving the bug from source inspection alone.

Phase 5 converted method into a trusted instrument. Several integrity bugs were caught before they could corrupt a reported number: a patch-sanitizer that silently truncated diffs ending on a blank context line; an oracle-gutting error where replication lag was injected on the wrong stream direction; a classifier that scored what models *said* rather than what they *did*; and a save-time corruption that a retroactive sweep over all 58 historical patches confirmed had damaged zero consequential fixes. The recurring lesson — an instrument that looks green is not green until the full corpus is run under the actual conditions — is why every Phase 6 number is oracle-graded and every cell ran through the identical validated pipeline.

4. Phase 6 Results

4.1 The headline: a gradient that inverts

Under identical frozen conditions, per-cell oracle resolution rates were:

Model	Vendor	Tier	Resolution rate
GPT-5.5	OpenAI	Frontier	100% (5/5)
Opus 4.8	Anthropic	Frontier	40% (2/5)
GPT-4.1	OpenAI	Mid-tier	40% (2/5)
Haiku 4.5	Anthropic	Mid-tier	0% (0/5)
Gemini 2.5 Pro	Google	Mid-tier	0% (0/5)

Read as a tier gradient (Figure 2), the extremes behave as expected: two mid-tier models floor at 0% and a frontier model saturates at 100%. But the middle breaks the story. **GPT-4.1, a mid-tier model, resolves at 40% — tying the frontier model Opus and sitting above two of its own mid-tier peers.** The tier label does not cleanly separate the models; the mid-tier band alone spans 0% to 40%, and its top overlaps the frontier. We do not headline the pooled rate ($9/25 = 36\%$): it mixes non-exchangeable tiers and is meaningless as a benchmark score.

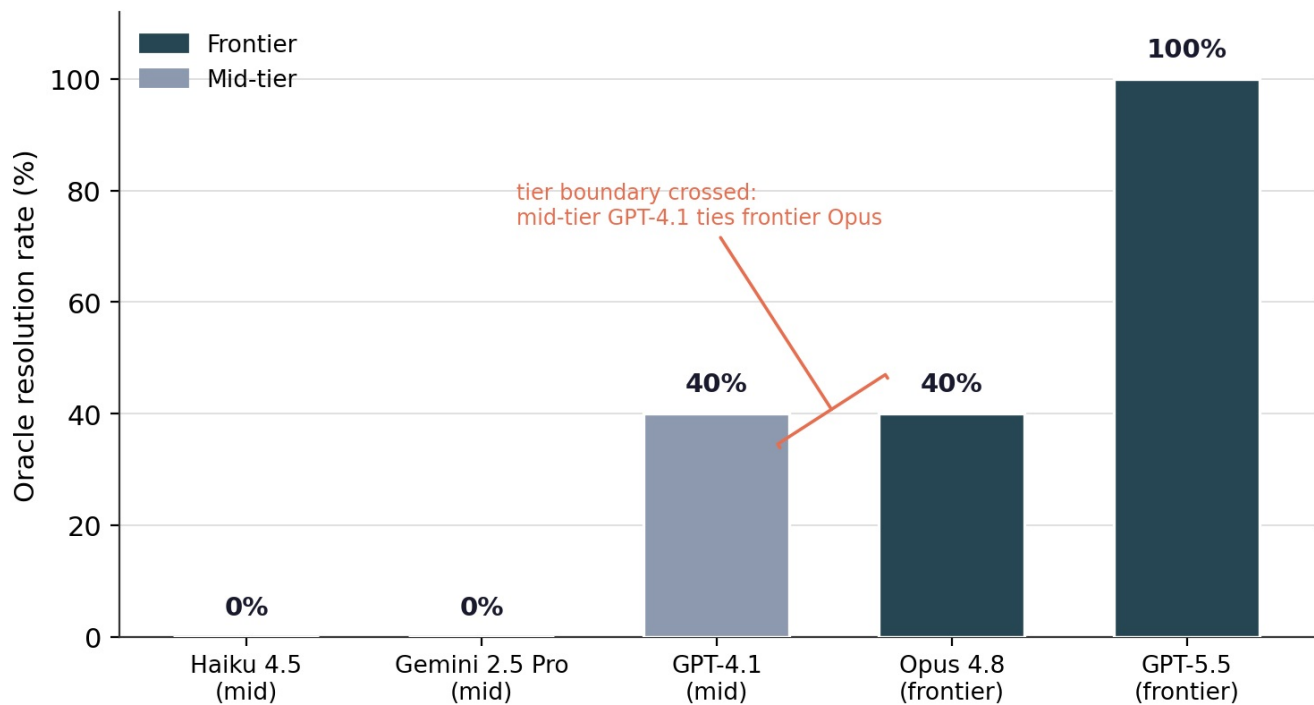


Figure 2. Oracle resolution rate by model, colored by tier. The gradient holds at the extremes but inverts in the middle: mid-tier GPT-4.1 (grey) ties frontier Opus (dark) at 40%, crossing the tier boundary.

4.2 Mechanism predicts what tier does not

Patch-level inspection of all 25 runs resolves the inversion and is the paper's central result (Figure 4). Grouping every run by the *mechanism the model actually implemented*: all nine passing runs — spread across four different models and both tiers — used exactly one of three sound mechanisms (always-read-primary, 4 runs; post-commit WAL-LSN capture, 3 runs; value-watermark catch-up, 2 runs). Every failure either produced no correct mechanism (wrong/unrelated fix, sleep band-aid, or no patch) or reached for the canonical LSN fix and got one ordering detail wrong.

Success tracks the **mechanism**, not the tier. GPT-4.1 reached 40% precisely because two of its runs used the blunt always-primary mechanism — the same mechanism behind one of Opus's two passes. It did not match a frontier model by being secretly frontier-capable; it matched by reaching for a simple correct fix instead of a sophisticated broken one.

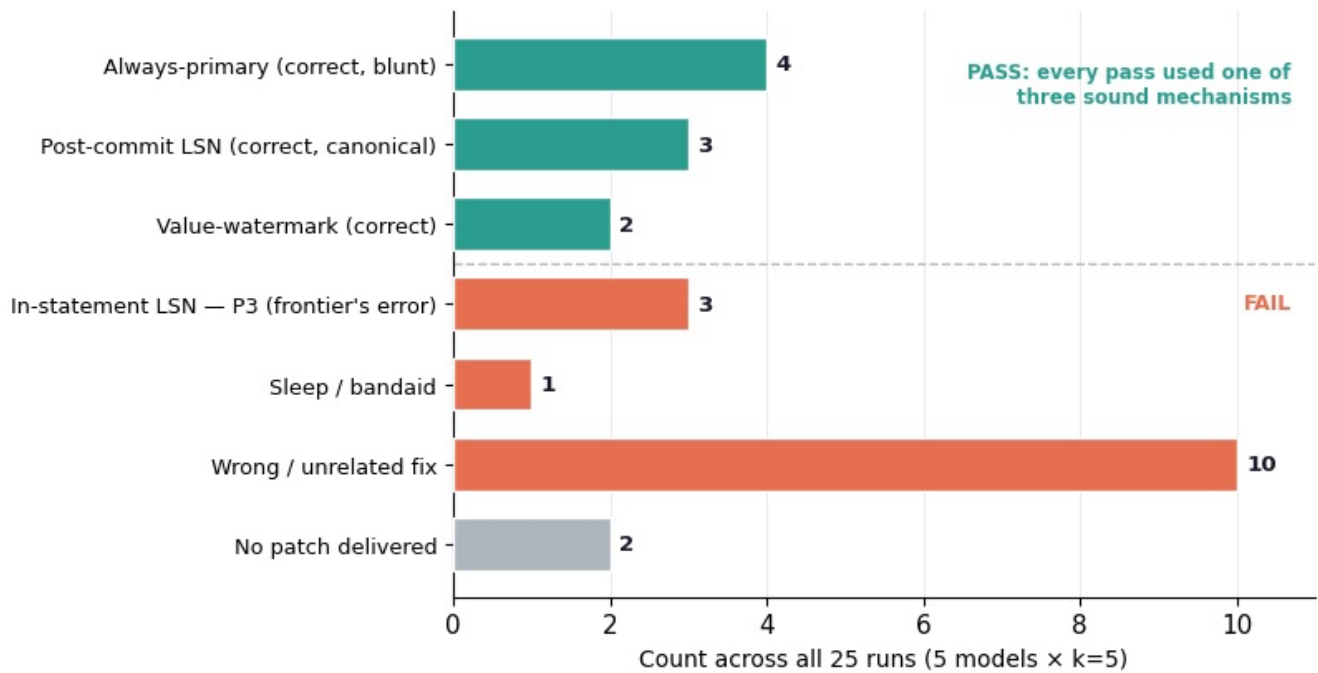


Figure 4. Every run across all 25, grouped by the mechanism implemented. All nine passes (green) used one of three sound mechanisms, spanning both tiers. In-statement LSN capture — the frontier's sophisticated error — sits on the fail side.

4.3 The frontier split: sophistication mis-executed

The 60-point gap between the two frontier models has a single, confirmed mechanistic cause (Figure 5). Both GPT-5.5 and Opus reached for the canonical LSN mechanism on several runs, but captured the write's WAL position differently. **GPT-5.5** captured it *after* the write committed, in a separate `SELECT pg_current_wal_lsn();` the gate then correctly holds reads on the primary until the replica has replayed the write — all three LSN patches pass (3/3). **Opus** captured it *inside* the writing statement via `UPDATE ... RETURNING ..., pg_current_wal_lsn();` evaluated mid-statement, this returns a pre-commit watermark the replica has typically already replayed, so the freshness gate is trivially satisfied and reads fall through to the stale replica — all three LSN patches fail (3/3), confirmed by direct inspection and reproduced deterministically by a controlled fixture.

This is the same in-statement-capture error the project isolated earlier as the “P3” defect, now confirmed on both sides of the split. The frontier model's failures are not random incapacity — they are its *sophistication mis-executed*. Opus's two passes avoid LSN entirely, using the same blunt-correct families (value-watermark, always-primary) that lower-tier passes relied on.

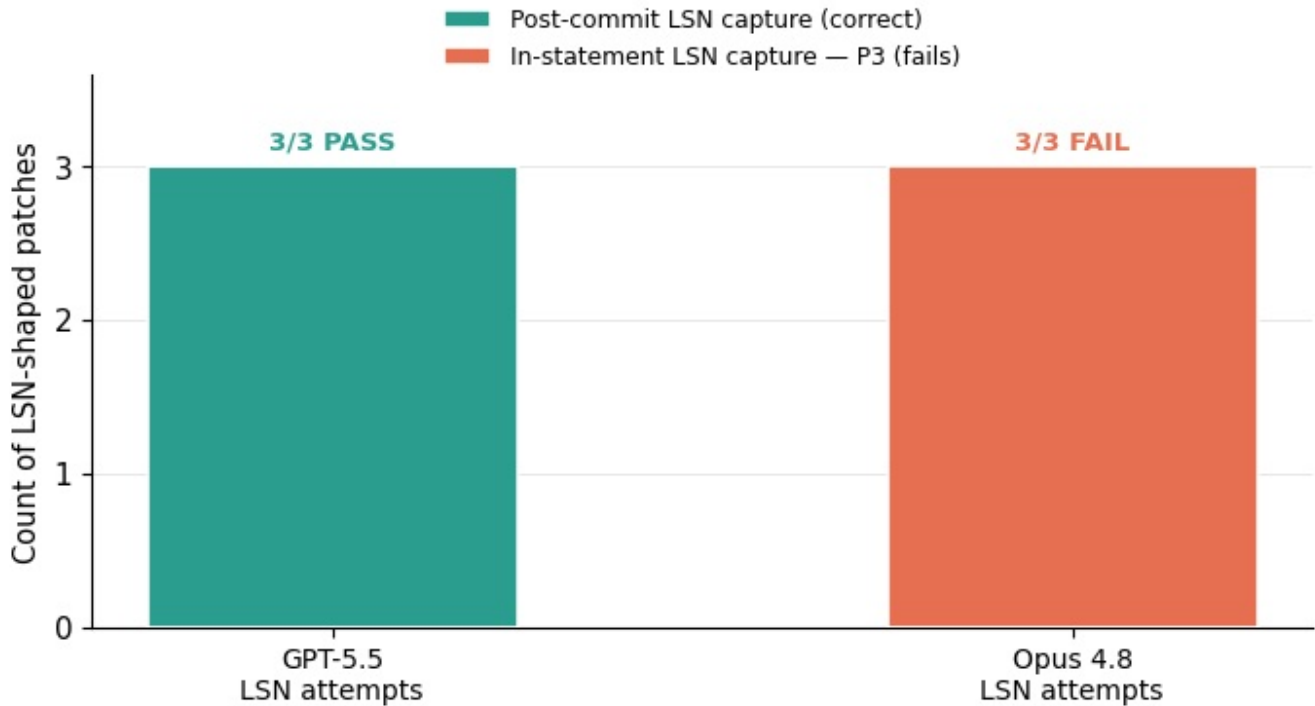


Figure 5. LSN-shaped patches from the two frontier models. GPT-5.5 captures the WAL position post-commit (correct, 3/3 pass); Opus captures it in-statement, a pre-commit watermark (the P3 defect, 3/3 fail). One ordering detail explains the frontier gap.

4.4 Mid-tier failure modes are not uniform

The two 0% cells fail differently (Figure 3). Gemini 2.5 Pro is a **pure capability floor**: it delivered a patch on all five runs and every one failed the oracle. Haiku 4.5 is a **mixed floor**: three delivered-but-wrong fixes and two runs with no patch. GPT-4.1’s non-zero cell is itself mixed — two passes, two capability fails, one build fail. “Mid-tier” is not a single behavior.

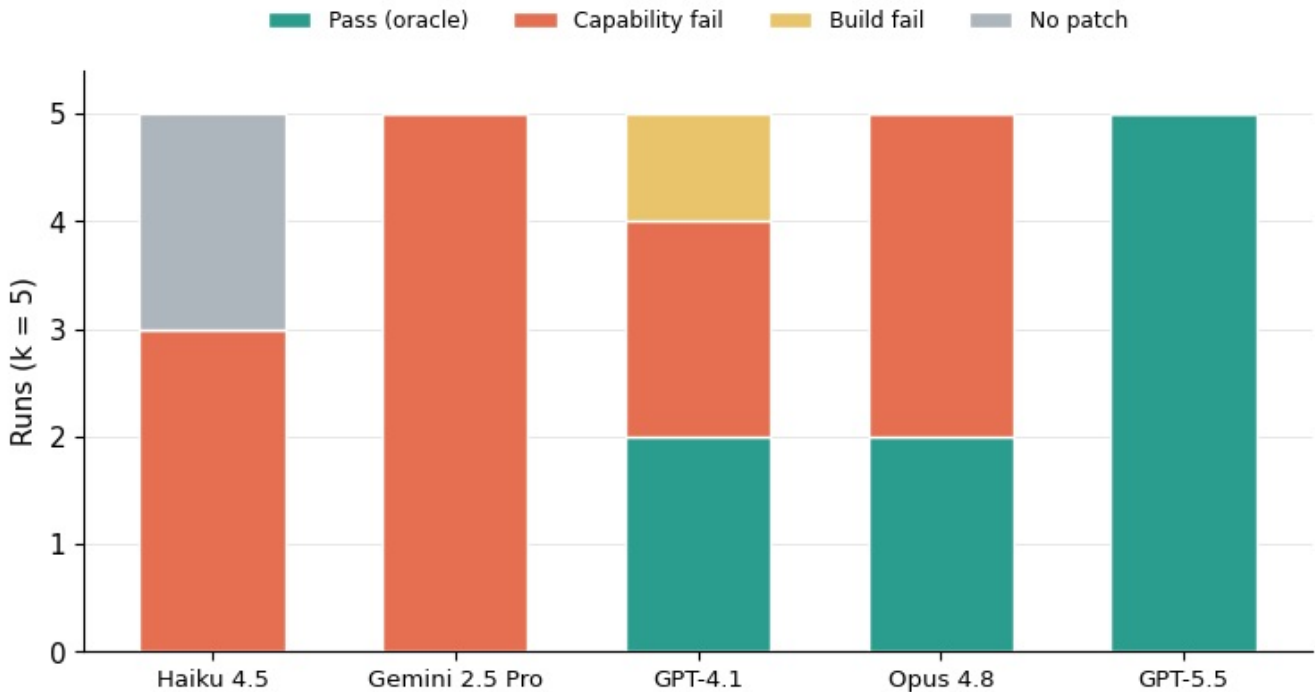


Figure 3. Outcome composition per model (k=5). The two floors differ: Gemini delivers five wrong patches (pure capability), Haiku mixes wrong fixes with non-delivery.

5. Validity and Scope

Supported. On the stale-read archetype under this frozen recovering-lag setup: (1) resolution success is predicted by the fix mechanism a model implements, not by its vendor tier, with a mid-tier model tying a frontier model and the frontier’s own failures traced to a specific mis-implementation; (2) the GPT-5.5 vs. Opus gap on LSN-shaped attempts maps to one confirmed transaction-ordering detail (post-commit vs. in-statement WAL capture), verified by patch inspection on both sides.

Not supported. No general ranking of model families on distributed debugging; no claim that any model “cannot” fix stale-read (Opus and GPT-4.1 both pass via alternate mechanisms); no conclusions about Gemini 3.1 (held out for delivery, not scored). This is one bug, one mechanism family, k=5 per cell.

Ceiling limitation. GPT-5.5 saturates this archetype at 5/5, so stale-read has no discriminating power above its capability level; a harder archetype would be needed to separate models beyond it. The archetype cleanly discriminates mid-tier from frontier at the extremes and separates the two frontier models by mechanism, but its ceiling is now reached.

6. Conclusion

GordianBench set out to measure whether agents genuinely repair distributed-systems bugs or merely suppress their symptoms, using a chaos oracle that rejects timing-luck fixes. Its Phase 6 sweep produces a result more interesting than the tier gradient it was scoped to find: on this archetype, *what a model attempts* predicts success better than *what tier it belongs to*. A mid-tier model ties a frontier one; the frontier model’s failures come from reaching for the canonical mechanism and mis-ordering a single WAL capture. The measurement protocol — oracle-graded, chaos-validated, pre-registered, and confirmed at the patch level — is what makes that finding trustworthy rather than an artifact of scoring. Extending the benchmark with harder archetypes that discriminate above the current frontier ceiling is the natural next step.

Appendix A. Frozen Configuration (Table 1)

Parameter	Value
Chaos profile	Recovering lag, 20,000 ms downstream WAL toxic
Oracle trials per patch	10 (PASS = 0 stale reads)
Runs per cell (k)	5
Turn budget	20
build_check	On (max 2 retries)
Prompt	Shared SYSTEM / USER, identical across cells
Scoring	Oracle verdicts only; classifier excluded
Rerun policy	Infra failures only; capability/build/no-patch as-is
Held out	Gemini 3.1 Pro (delivery non-comparability)

Appendix B. Confirmed Mechanistic Split

GPT-5.5 (passing) captures the watermark **after** commit; Opus (failing) captures it **in-statement**, before commit:

```
GPT-5.5 PASS:                               Opus 4.8 FAIL (P3):
UPDATE accounts SET ...                     UPDATE accounts SET ...
  RETURNING balance_cents;                  RETURNING balance_cents,
-- separate, after commit:                  pg_current_wal_lsn()::text;
SELECT pg_current_wal_lsn();               -- in-statement: pre-commit watermark
-- post-commit watermark -> PASS           -- gate trivially true -> stale -> FAIL
```

Appendix C. Artifact Index

Phase 4 canonical: archetype-a/phase_4_report_6.md. Phase 5 gating: phase_5_report_13.md, PRE_PHASE6.md. Phase 6 canonical: phase_6_report.md, phase6_cells/phase6_headline_rollup.json. Mechanistic diffs: phase6_cells/gpt-5.5/PARTIAL_PASS_DIFF.md, phase6_cells/opus-4.8/OPUS_P3_DIFF.md.